

Name: Sreeja Reddy Yeluru

Comments on DRL Agent After Learning

Training Duration and Process:

Training time for the deep Q-network (DQN) agent to learn atari pong game took about 2.5 to 3 hours to train over 2.5 million steps from 700 thousand steps. With all of the changes that were made during the second round of testing, total time spent to test was about 6-7 hours. Each checkpoint was taken every 300 thousand steps and included both the model weights (q_net and target_net) as well as the replay buffer and saved to google drive.

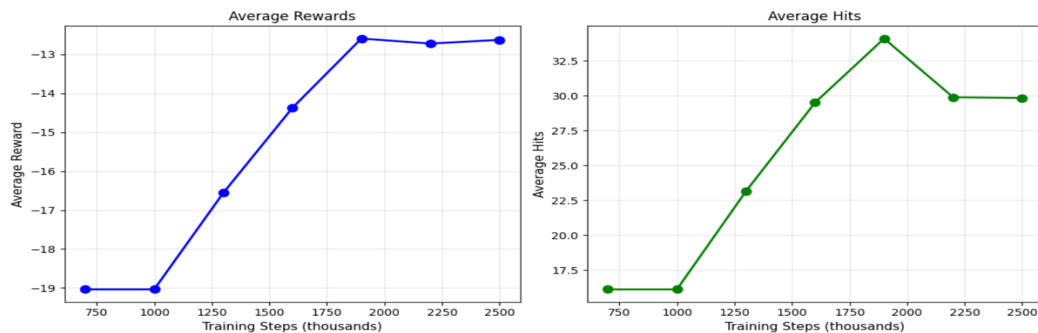
The baseline 700k-1M training took 30-35 minutes, producing an average reward of -19.04 with 16.11 hits per episode. The subsequent training showed clear progression:

- 1M to 1300k : -16.56 reward, 23.15 hits
- 1300k to 1600k: -14.38 reward, 29.50 hits
- 1600k to 1900k: -12.59 reward, with 34.09 hits(best performance)
- 1900k to 2200k: -12.72 reward, with 29.88 hits
- 2200k to 2500k: -12.62 reward, with 29.83 hits

Performance summary (700k - 2.5M)	
Checkpoint	Evaluation Reward
700k	-19.037037037037038
1000k	-19.037037037037038
1300k	-16.558823529411764
1600k	-14.375
1900k	-12.590909090909092
2200k	-12.72
2500k	-12.625

The same performance score of -19.04 appears for both 700k and 1m checkpoints, as a flat horizontal line. This is because these values came from a single continuous training run from 700k to 1m, which produced one averaged performance metric across that entire 300k step period. Therefore, the 700k value could only have been obtained if the training data from the preceding segment (400k-700k) were available; however, those training data were not. All subsequent checkpoints (from

1.3m to 2.5m) each represent 300k step training periods, and therefore their own performance averages



Trained Agent Played Well?:

Yes, the agent played much better than I had expected. Average rewards increased by 34%, from -19.04 to -12.59. Hits also increased more than double, from 16.11 to 34.09. At early checkpoints (700k-1M), the agent would frequently miss the ball or move improperly. However, by 1900k, the agent had a consistent ability to follow the ball and position the paddle in an appropriate manner, indicating intentional strategy and no longer simply moving randomly.

It is interesting to note that performance did decline somewhat after 1900k, leveling off at -12.6 to -12.7. This is characteristic of reinforcement learning, and performance will continue to fluctuate while the agent continues to explore strategies and update Q-values. This is in contrast to the typical monotonic improvement seen when using supervised learning.

Did It Exceed Expectations?:

Yes, they certainly were. I didn't know what to expect of dqn's performance on Pong, so I was pleasantly surprised when I went from losing every rally (-19.00) to consistently returning the ball (-12.60). My biggest surprise was the speed at which the agent progressed between 1M and 1.6M steps - the agent had a "breakthrough moment" where it determined the core strategy for winning rallies: track the ball and hit it with the paddle.

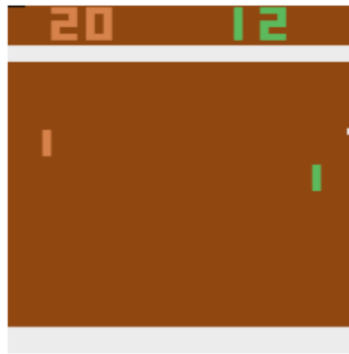
Observations on Agent Learning and Behavior:

I found generating videos of the agent's progression to be really interesting. I created videos from the agent's best-performing checkpoint (1.9M) and last checkpoint

(2.5M), which are located in the videos directory. The 1.9M checkpoint video illustrated many clearly learned behaviors:

- **Ball tracking:** The agent always placed the paddle such that it aligned with the trajectory of the ball, and it anticipated where the ball would be before it actually had passed by.
- **Positioning:** The agent would proactively place the paddle in an optimal position to intercept the ball based on its velocity and direction.
- **Recovery:** The agent rapidly returned to the center of the court after missing the ball or the opponent scoring, to prepare for the next rally

Generating video: 1900k checkpoint



WARNING:imageio_ffmpeg:IMAGEIO_FFMPEG_WR
Total reward: -9.0
Episode reward: -9.0

The 1900k video episode achieved -9.0 reward, which translates to the agent scoring 12 points while the opponent scored 21 - a much better performance than the -12.59 training average. This demonstrates that the agent had indeed learned effective strategies and could compete reasonably well, even though it didn't consistently win. This variability is normal in RL - individual episodes can vary significantly depending on opponent patterns and random factors. The fact that the agent could achieve -9.0 and score 12 points in a single game shows it had learned to play competently, even if not applied consistently every game.

General Reflections

- Deep Reinforcement Learning taught me many important things about how experience replay and target networks work in practice - the replay buffer keeps from having correlated samples in the sample set and target networks that are updated every 10,000 steps give stable Q-value targets for use during the training process. Deep RL does not show immediate progress as Supervised Learning will - it takes hundreds of thousands of steps to see real changes. Using a replay buffer and checkpointing every 300,000 steps has

been incredibly helpful in tracking progress and in case something goes wrong in the training process.

- I would say this project was a medium to hard difficulty level. The easy part was getting the training loop implemented and up running once I knew what needed to be done and creating the videos and charts to track the agents' performance using the helper functions provided. The harder part was figuring out how all the components of DQN worked together, and debugging when the agent wouldn't learn from its experiences, and keeping track of the training process that ran for 2-3 hours each time and making sure the checkpoints were written correctly.
- The hardest part was managing the length of time the training runs were taking and debugging problems that required a complete new start. The first training process took 2-3 hours to complete 700,000 to 2.5 million steps of training, but by the time I went back to look at the performance chart, there was no data point at 2.5 million steps. I found out later that I did not write the performance data to the tracking arrays after the final iteration of training; therefore, I had to go into the code and correct the problem with the way the arrays were aligned and then I had to run the training process again completely from the beginning, and that took another 2-3 hours. That resulted in a total of 6-7 hours of training time.
- If I were to do this again, I would make sure that I verified my code to track the performance of the agent in a small test run (100k-200k steps) to make sure that the arrays are being updated at each step of the training process before I committed to a full-length training run that would take 2-3 hours. I would also create an automated system for saving the check points for the training process, one that includes not only the model weights but the current state of all tracking arrays. If I had a failure during the training process, such as losing power or discovering an error, I would have the option to restart the training process from the last successful checkpoint instead of having to restart from the very beginning.
- I found this assignment to be both difficult and rewarding, as it gave me a hands on opportunity to learn and apply Deep Reinforcement Learning concepts instead of just reading about them. The most rewarding moment was

viewing the generated video of the agent playing Pong and watching it perform well based on its ability to learn from millions of steps of experience.

References: Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. A. (2013). Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*.